Automatic Data Curation for the Next Generation of Radio Science Experiments

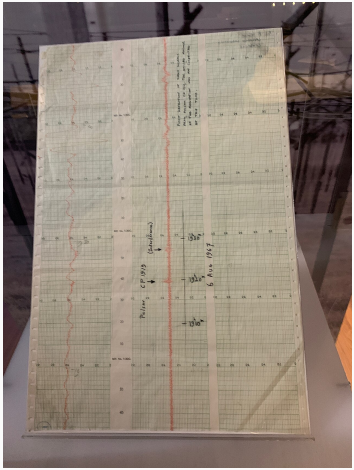
Sam Leeney

University of Cambridge

With: Will Handley, Harry Bevins, Eloy de Lera Acedo

sakl2@cam.ac.uk

The first pulsar: found by eye



- Jocelyn Bell Burnell, 1967: the first pulsar detected by a human anomaly flagger.
- Humans are still partially involved in data curation and cleaning today.
- Data rates now reach TB to PB scales.
- This talk: fully automated data curation. No human in the loop. No hyperparameters to tune.

Outline

Method

Results on REACH

Results on Type Ia Supernovae

Results in Transient Searching

Method

Why not just flag first and fit later?

Standard workflow

Run anomaly detection before inference.

Tune thresholds or hyperparameters.

Manually inspect edge cases.

Throw the flagged data away.

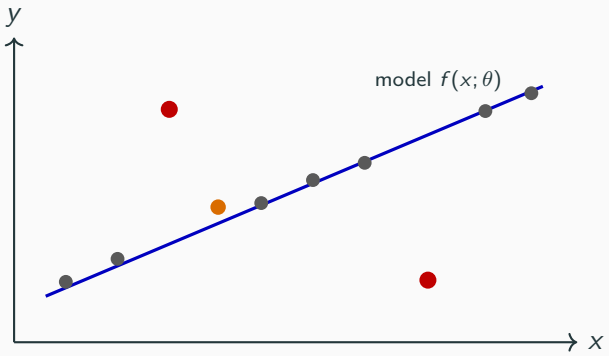
Why this is a problem

Data curation is decoupled from the physics model, so useful information can be deleted, the workflow is rarely fully automatic, and humans in the loop are prone to bias. We want the model of the thing we understand to inform the model of the thing we don't.

$\Longrightarrow$

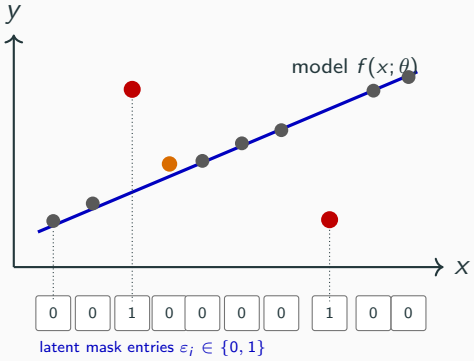
Put the data-quality model inside the fit.

What if the data contain contaminants?



Start with a model and two contaminants. We want to fit the line and flag the bad points in one go.

What if the mask is part of the model?



Assign each data point a label: $\varepsilon_i = 0$ (nominal) or $\varepsilon_i = 1$ (anomalous). This mask becomes part of the parameter vector.

How do we find the right mask?

We take a Bayesian approach: marginalise over **all possible masks** to get a probability per point that it is contaminated.

Write the likelihood blending normal and anomaly terms:

$$P(\mathcal{D}, \vec{\varepsilon} \mid \theta) = \prod_i \left[(1 - p_i)\, \mathcal{L}_i(\theta)\right]^{1-\varepsilon_i} \left[\frac{p_i}{\Delta}\right]^{\varepsilon_i}$$

In principle we sum over all 2^N mask configurations. This quickly becomes intractable.

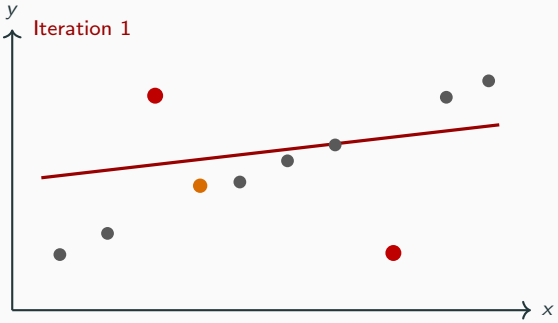
Dominant mask approximation

Instead of summing, we **maximise**: at each step of the sampler, pick the best mask for the current θ. Because the pixels factorise, this reduces to a per-point comparison:

$$\varepsilon_i^\star = 1 \quad \text{if} \quad \frac{p_i}{\Delta} > (1 - p_i)\, \mathcal{L}_i(\theta)$$

What does this look like in practice?

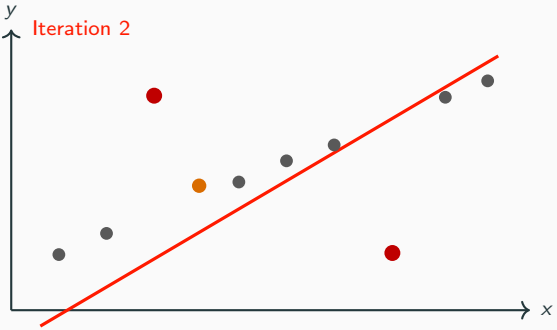
What does this look like in practice?



1	**1**	0	0	0	0	0	**1**	0	**1**

mask $\vec{\varepsilon}$: compare $(1-p_i)\mathcal{L}_i(\theta)$ vs p_i/Δ for each point

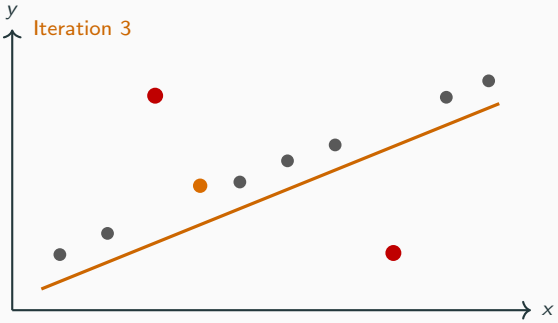
What does this look like in practice?



0	0	**1**	0	**1**	0	0	**1**	**1**	**1**

mask $\vec{\varepsilon}$: compare $(1-p_i)\mathcal{L}_i(\theta)$ vs p_i/Δ for each point

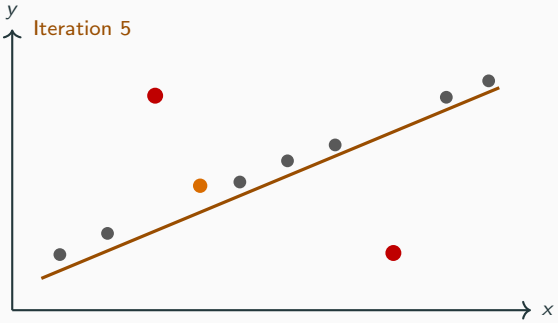
What does this look like in practice?



0	0	**1**	**1**	0	0	0	**1**	0	0

mask $\vec{\varepsilon}$: compare $(1-p_i)\mathcal{L}_i(\theta)$ vs p_i/Δ for each point

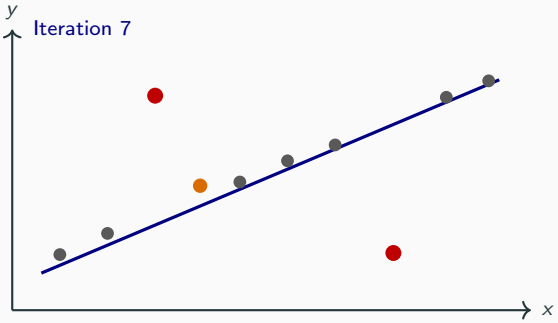
What does this look like in practice?



0	0	**1**	**1**	0	0	0	**1**	0	0

mask $\vec{\varepsilon}$: compare $(1-p_i)\mathcal{L}_i(\theta)$ vs p_i/Δ for each point

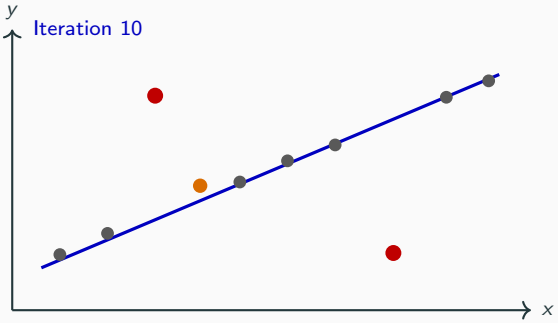
What does this look like in practice?



0	0	**1**	0	0	0	0	**1**	0	0

mask $\vec{\varepsilon}$: compare $(1-p_i)\mathcal{L}_i(\theta)$ vs p_i/Δ for each point

What does this look like in practice?



0	0	**1**	0	0	0	0	**1**	0	0

mask $\vec{\varepsilon}$: compare $(1-p_i)\mathcal{L}_i(\theta)$ vs p_i/Δ for each point

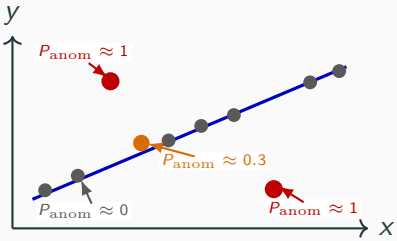
Weighted average over samples gives probabilities

	x_1	x_2	x_3	x_4	x_5	x_6	x_7	x_8	x_9	x_{10}	wt
$\bar{\varepsilon}^{(1)}$:	1	1	0	0	0	0	0	1	0	1	.02
$\bar{\varepsilon}^{(3)}$:	0	0	1	1	0	0	0	1	0	0	.13
$\bar{\varepsilon}^{(10)}$:	0	0	1	0	0	0	0	1	0	0	.85
P_{anom}:	**.02**	**.02**	**.98**	**.13**	**0**	**0**	**0**	**1**	**0**	**.02**	

The resulting log-likelihood with the dominant mask:

$$\log P = \sum_i \max\{\log \mathcal{L}_i + \log(1-p_i),\ \log p_i - \log \Delta\}$$

Note: $\log \Delta$ acts as an **Occam penalty**. Flagging a point costs $\log \Delta$; the model only flags when the likelihood gain outweighs this cost. The threshold p_i is **fit as a model parameter**, not set by hand.



A familiar activation function

Take logs of the per-point decision rule from our likelihood:

$$\frac{p_i}{\Delta} > (1 - p_i)\,\mathcal{L}_i \qquad \Longrightarrow \qquad \log p_i - \log(1 - p_i) > \log \mathcal{L}_i + \log \Delta$$

$$\boxed{\mathrm{logit}(p_i) > \log \mathcal{L}_i + \log \Delta}$$

Invert with the sigmoid $\sigma(x) = 1/(1 + e^{-x})$ to get the posterior anomaly probability:

$$P_{\mathrm{anom},i} = \sigma\big(\mathrm{logit}(p_i) - \log \mathcal{L}_i - \log \Delta\big)$$

The logit and sigmoid fall directly out of our likelihood. These are standard activation functions in machine learning. The same framework that works inside nested sampling can therefore be made differentiable and plugged into a neural network.

Hard vs. soft mixture models

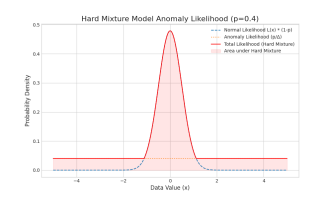
Hard mixture: Bayesian inference

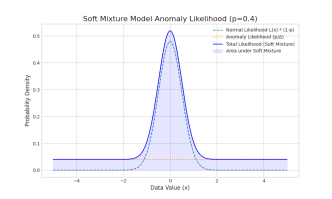
$$\log p(\mathcal{D} \mid \theta) = \sum_i \max\{\log \mathcal{L}_i + \log(1 - p_i),\ \log p_i - \log \Delta_i\}$$

Soft mixture: machine learning

$$\log p(\mathcal{D} \mid \theta) = \sum_i \log \left[(1 - p_i)\mathcal{L}_i + \frac{p_i}{\Delta_i}\right]$$

The hard max becomes a differentiable log-sum-exp. The corner between the flat floor and the likelihood has no gradient; the soft version smooths this, letting the same idea move into gradient-based machine learning.





Read the papers

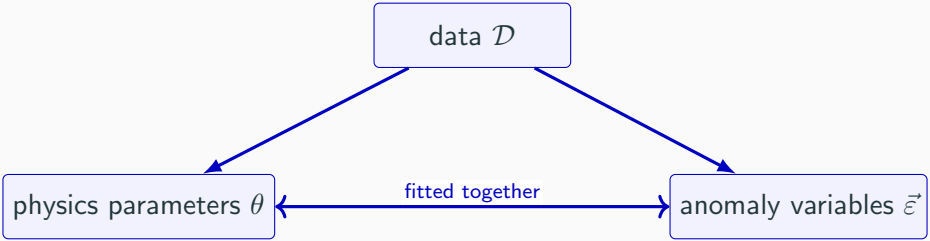
Leeney, Handley, de Lera Acedo (2023). A Bayesian approach to RFI mitigation. arxiv: 2211.15448

Anstey, Leeney, Handley, de Lera Acedo (2024). Enhanced Bayesian RFI Mitigation and Transient Flagging. arxiv: 2310.02146

Leeney, Handley, Bevins, de Lera Acedo (2025). Bayesian Anomaly Detection for Ia Cosmology: Automating SALT3 Data Curation. arxiv: 2509.13394

Leeney, Handley, Bevins, de Lera Acedo (in prep). Automatic Data Curation for the Next Generation of Radio Science Experiments.

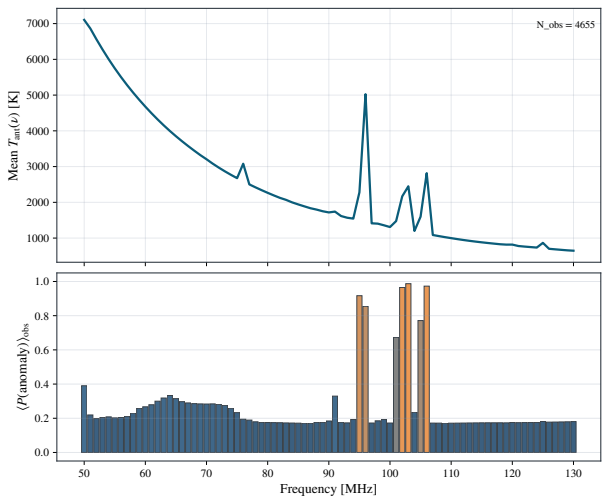
What is actually being fitted?



This is the key point: the anomaly fit is done **simultaneously** with the model fit. Nothing is pre-flagged, and anomalous data are downweighted rather than thrown away. Can be implemented with 2 lines of code.

Results on REACH

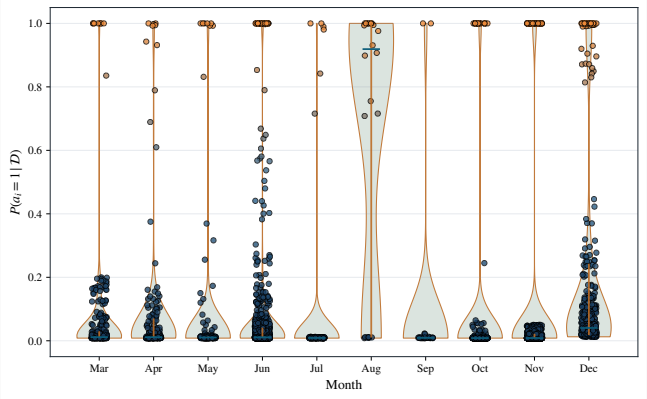
What does the model see in a year of REACH data?



- A year of REACH data organised by LST: the galaxy moving overhead.
- We jointly fit the whole year simultaneously.
- Each observation gets a curation **score**, fit simultaneously with the physics.

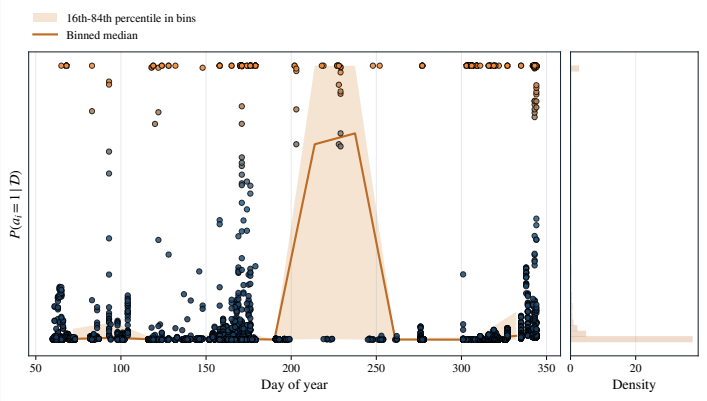
Fully automated: no manual curation pass. Bottom panel: score per frequency channel, marginalised over the year. Channels with $P_{\rm anom} \approx 1$ are excluded automatically.

What if we slice the scores by month?



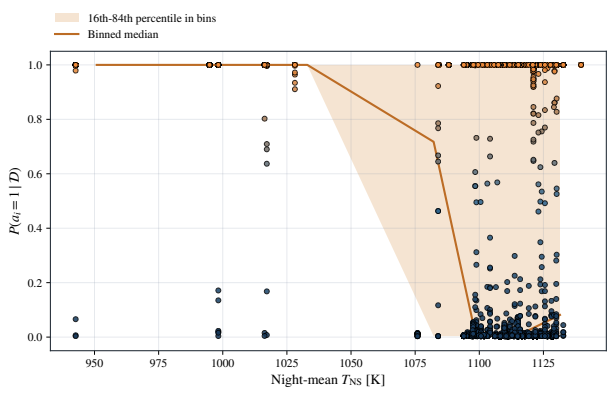
Once the joint fit is done, we can examine the curation score along any axis of interest. Here the same automatic score is sliced by month.

What if we slice by day of year?



These observations were automatically downweighted during inference. There was no separate manual curation pass.

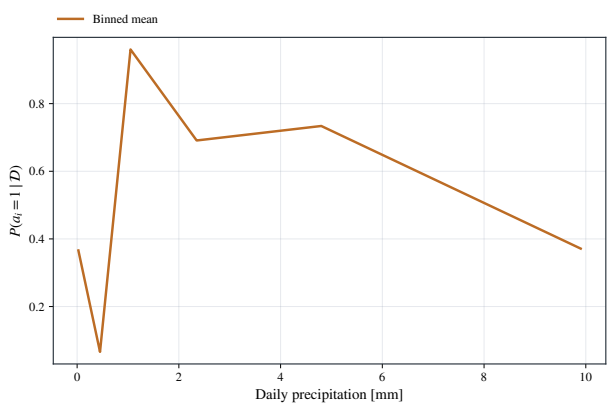
Does the model find instrument problems by itself?



REACH is calibrated with internal noise sources. One important source, labelled TNS here, should sit near 1100 K.

The flagger does not see TNS. It finds bad data, then after the fact we notice this correlates to the temperature of an internal component.

Does the model recover weather without weather inputs?

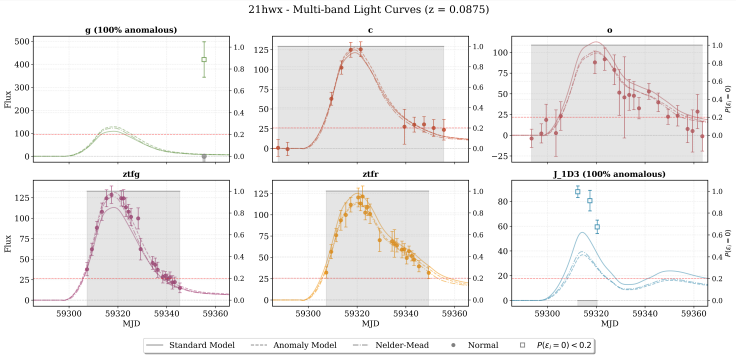


The anomaly model does not know anything about precipitation.

Even so, it assigns poor scores to wet days. Our telescope now doubles as a weather station. The interesting point is that the same model is also sensitive to subtler contamination patterns that a simple weather cut would miss.

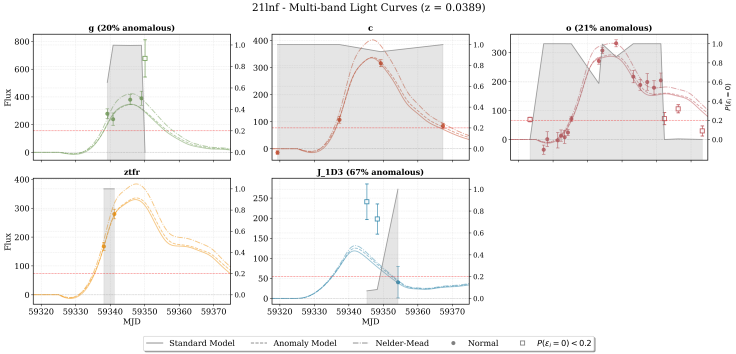
Results on Type Ia Supernovae

Can the same model switch off broken filters?



SN 21hwx: on some days, we know certain filters were broken. The flagger automatically identifies that the entire ZTF g and j bands are problematic and flags them. Previously this would have been done by hand.

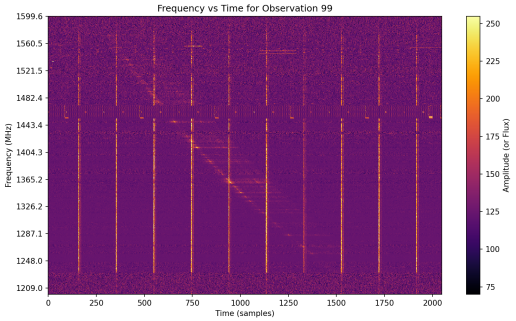
Can it also save data that humans would discard?



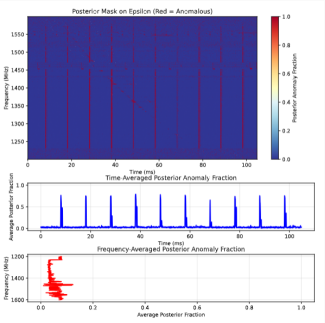
SN 21lnf: rather than discarding an entire band, the flagger selectively removes individual outliers. The 1dj3 band is **saved**: a human would have thrown the whole band away, but the model keeps the good points.

Results in Transient Searching

Does this scale to transient searches?



Real data



Posterior anomaly mask

The same method curates each pixel. In the marginalised plots we see it correctly identifies channels with RFI, without a separate hand-built curation stage.

Summary

1. **Fully automated anomaly flagging** inside the fit
2. **Fully automated data curation** with no manual good/bad selection
3. **Fully automated data cleaning** that can also save data humans would have cut

The same idea now runs across Bayesian inference, machine learning, REACH analysis, supernova light curves, and transient searching.

Thanks for listening!

Questions?

Slides and source:

github.com/samleeney/Talks

Email:

sakl2@cam.ac.uk

QR code to the talk repository

Appendix

Full derivation: Bayesian anomaly detection

1. Likelihood per data point under the model: $\mathcal{L}_i(\theta)$.

2. Introduce a Bernoulli mask $\varepsilon_i \in \{0, 1\}$ with prior anomaly rate p_i and anomaly scale Δ:

$$P(\mathcal{D}, \vec{\varepsilon} \mid \theta) = \prod_i \left[(1 - p_i)\, \mathcal{L}_i(\theta)\right]^{1-\varepsilon_i} \left[\frac{p_i}{\Delta}\right]^{\varepsilon_i}$$

3. Marginalise the mask. Pixels factorise, so the 2^N sum is tractable in principle:

$$P(\mathcal{D} \mid \theta) = \prod_i \left[(1 - p_i)\, \mathcal{L}_i + \frac{p_i}{\Delta}\right]$$

4. Dominant mask approximation (maximise instead of sum):

$$\log P \approx \sum_i \max\{\log \mathcal{L}_i + \log(1 - p_i),\ \ \log p_i - \log \Delta\}$$

5. Posterior anomaly probability per point (Bayes' rule on mask states):

$$P_{\mathrm{anom},i} = \frac{p_i/\Delta}{(1 - p_i)\, \mathcal{L}_i + p_i/\Delta}$$

Full derivation: logit and sigmoid

6. Take logs of the per-point decision rule:

$$\frac{p_i}{\Delta} > (1 - p_i)\,\mathcal{L}_i \quad\Longrightarrow\quad \log\frac{p_i}{1-p_i} > \log\mathcal{L}_i + \log\Delta \quad\Longrightarrow\quad \mathrm{logit}(p_i) > \log\mathcal{L}_i + \log\Delta$$

7. Rearrange $P_{\mathrm{anom},i}$ by dividing top and bottom by p_i/Δ:

$$P_{\mathrm{anom},i} = \frac{1}{1 + \frac{(1-p_i)\,\mathcal{L}_i\,\Delta}{p_i}} = \frac{1}{1 + \exp\big(\log\mathcal{L}_i + \log\Delta - \mathrm{logit}(p_i)\big)} = \sigma\big(\mathrm{logit}(p_i) - \log\mathcal{L}_i - \log\Delta\big)$$

where $\sigma(x) = 1/(1 + e^{-x})$ is the sigmoid function.

8. Soft mixture (ML extension): replace max with $\mathrm{logsumexp}$:

$$\log P(\mathcal{D} \mid \theta) = \sum_i \log\left[(1 - p_i)\,\mathcal{L}_i + \frac{p_i}{\Delta}\right]$$

Leeney+23 2211.15448 · **Anstey+24** 2310.02146 · **Leeney+25** 2509.13394 · **Leeney+** (in prep)